Machine Learning Operations (AIMLCZG523) — Assignment 1

Heart Disease Risk Prediction: An End-to-End MLOps Pipeline

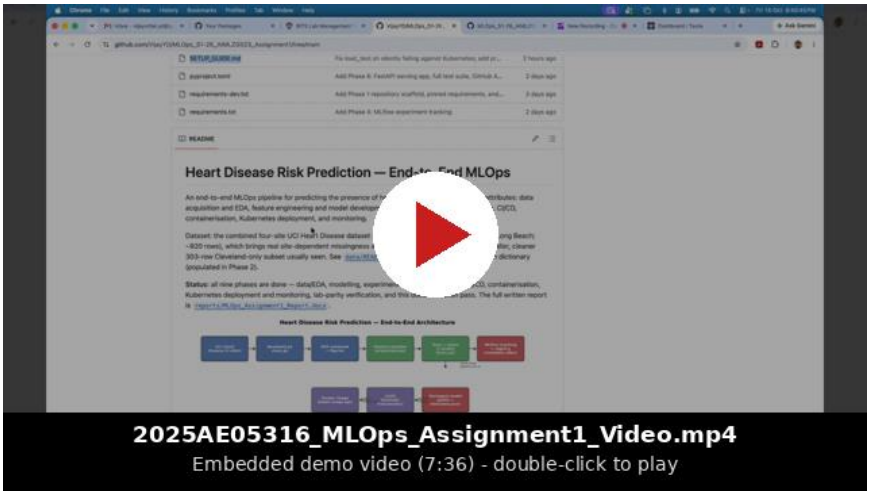
STUDENT NAME: YELDONA DESAI V VITTAL

STUDENT ID: 2025AE05316

2026-10-16

2025AE05316_MLOps_Assignment1_GitHub_Link

Demo video (embedded): 2025AE05316_MLOps_Assignment1_Video.mp4



Double-click the thumbnail in Microsoft Word (desktop) to open and play the video.

Online Drive Link

Table of Contents

Table of Contents 1
1. Introduction and Problem Statement 3
2. Setup and Installation Instructions 3
Mac / development 3
Lab (Linux) 3
3. Data Acquisition and Exploratory Data Analysis 4
4. Feature Engineering and Modelling 7
Leave-one-site-out generalisation 11
5. Experiment Tracking 12
6. Model Packaging and Reproducibility 13
7. CI/CD Pipeline and Automated Testing 14
8. Containerisation and API Design 15
9. Kubernetes Deployment 17
10. Monitoring and Logging 18
11. Architecture Diagram 19
12. Limitations, Ethical Considerations, and Future Work 19
13. References 20

1. Introduction and Problem Statement

Heart disease remains a leading cause of death worldwide, and early identification of at-risk patients from routinely collected clinical attributes is a long-standing machine learning problem. This project builds a binary classifier that predicts the presence of heart disease from thirteen clinical attributes, and ships it as a containerised, monitored, Kubernetes-deployed API with a full CI/CD pipeline and experiment tracking.

Rather than use the common 303-row Cleveland-only subset of the UCI Heart Disease dataset, this project combines all four original sites (Cleveland, Hungarian, Switzerland, and VA Long Beach; 918 rows after cleaning). This is a deliberately harder and more realistic setting: the four sites differ substantially in missingness patterns and disease prevalence, which makes naive pooled evaluation optimistic about real-world generalisation to a new hospital. A leave-one-site-out evaluation, in addition to standard stratified cross-validation, is used throughout to make this risk visible rather than hidden.

Every non-obvious design decision in this project — from the dataset and missing-value handling to the model-selection rule and Kubernetes topology — is logged with its rationale and alternatives considered in docs/DECISIONS.md, referenced throughout this report by its decision ID (e.g. D3.1).

2. Setup and Installation Instructions

The project targets two environments with different Python versions: macOS with Python 3.13.x for development, and a Linux lab (BITS Prayogshala) with Python 3.12.9 for verification. scripts/setup_venv.sh resolves the correct pinned interpreter for the current OS rather than trusting whatever python3 happens to be first on $PATH, since the two can silently differ (see docs/DECISIONS.md D0.3).

Mac / development

bash scripts/check_env.sh

bash scripts/setup_venv.sh && source .venv/bin/activate

make data && make train

Lab (Linux)

LAB_RUNBOOK.md gives the full copy-paste sequence — clone, preflight, venv, data, training, tests, Docker, Kubernetes deploy/verify/load-test/teardown, and a troubleshooting section — verified by an actual fresh-clone dry run before being trusted (docs/DECISIONS.md D8.2). No step anywhere requires sudo.

3. Data Acquisition and Exploratory Data Analysis

src/heart_mlops/data/download.py downloads the four UCI site files with SHA-256 checksum verification, retries, and an automatic fallback to the UCI static archive if a primary URL is unreachable — never fabricating data if both sources fail. src/heart_mlops/data/clean.py then converts '?' tokens and physiologically impossible zero values (chol == 0 in all 123 Switzerland rows and 49 VA rows; trestbps == 0 in one VA row) to missing, removes two exact duplicate records, and binarises the target (target = 1 if num > 0 else 0). The cleaned file deliberately keeps every missing value — imputation happens only inside the model pipeline, fit on training folds only, so no information leaks from validation or test rows.

Site	Rows	trestbps % missing	chol % missing	slope % missing	ca % missing	thal % missing	Disease %
cleveland	303	0.0	0.0	0.0	1.3	0.7	45.9
hungarian	293	0.3	7.5	64.5	99.0	90.4	36.2
switzerland	123	1.6	100.0	13.8	95.9	42.3	93.5
va	199	28.6	28.1	50.8	99.0	82.9	74.4
ALL	918	6.5	21.9	33.4	66.3	52.7	55.3

Table 1: Rows, missing data (%) for the columns most affected, and disease prevalence (%), by site. Full per-column breakdown in data/processed/data_quality_summary.csv.

The missingness is not uniform: ca, thal and slope are missing in 60-99% of rows outside Cleveland, and disease prevalence swings from 36.2% (Hungarian) to 93.5% (Switzerland). Both are visible in the figures below and motivate the site-aware modelling choices in Section 4.

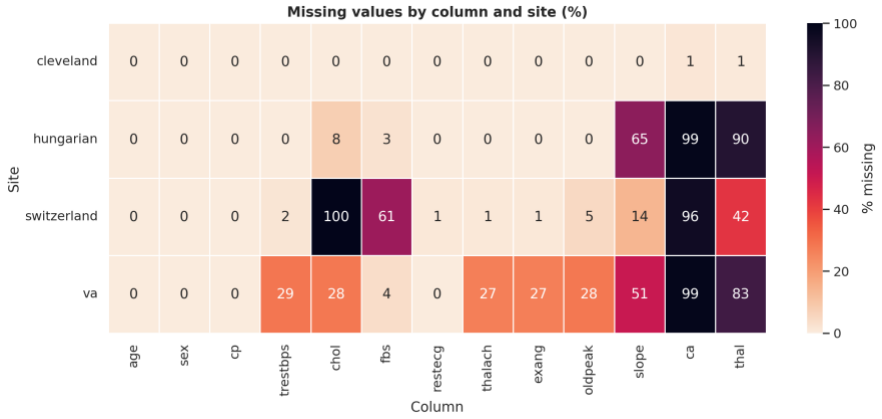


Figure 1: Missing values by column and site.

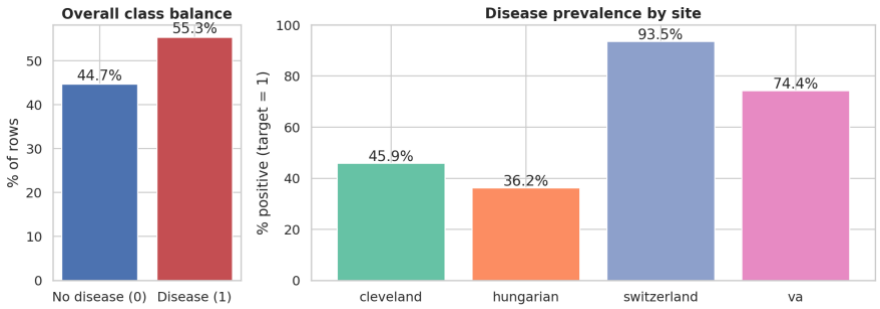


Figure 2: Overall and per-site class balance.

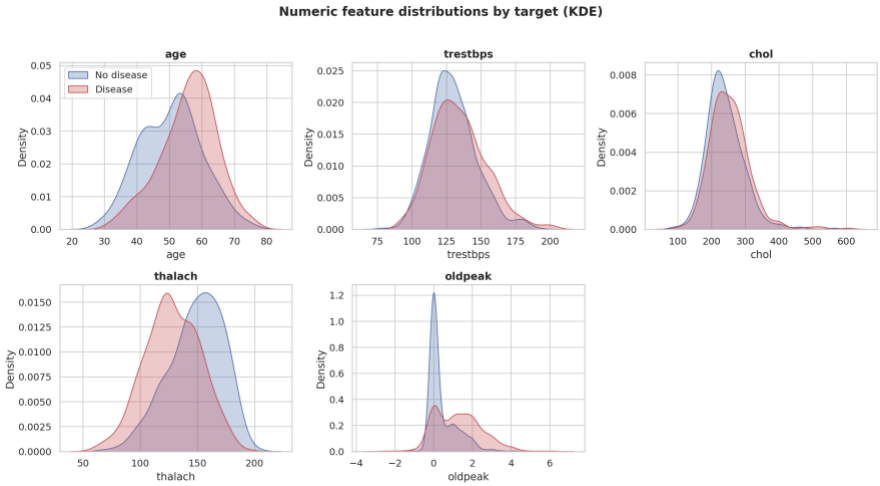


Figure 3: Numeric feature distributions by target (KDE).

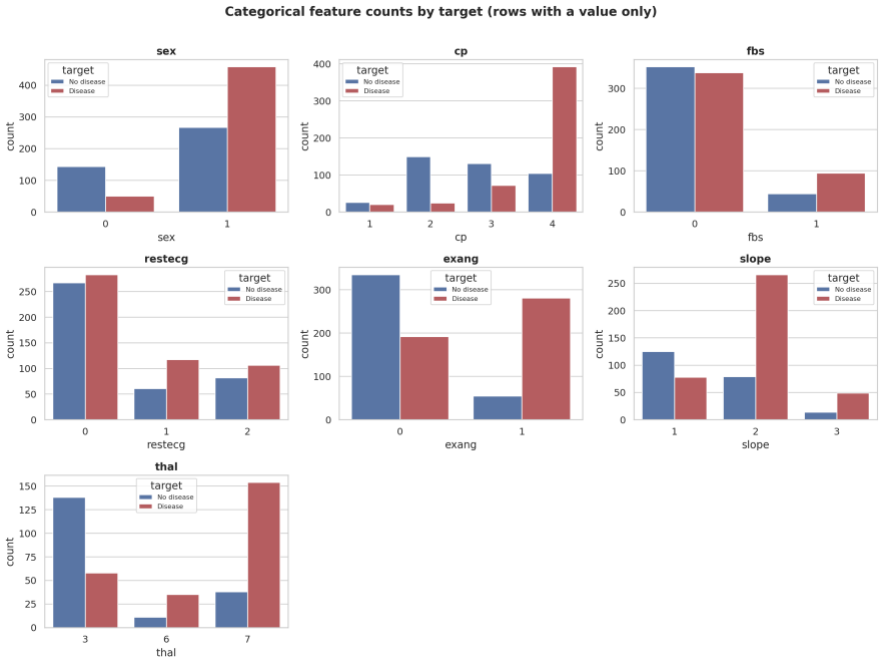


Figure 4: Categorical feature counts by target.

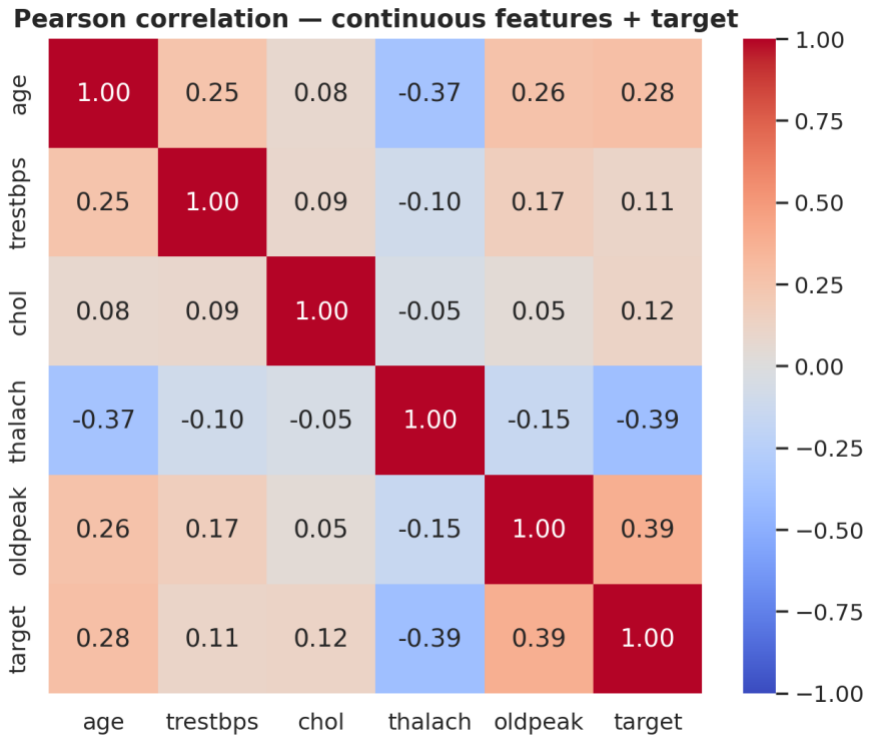


Figure 5: Pearson correlation among continuous features and target.

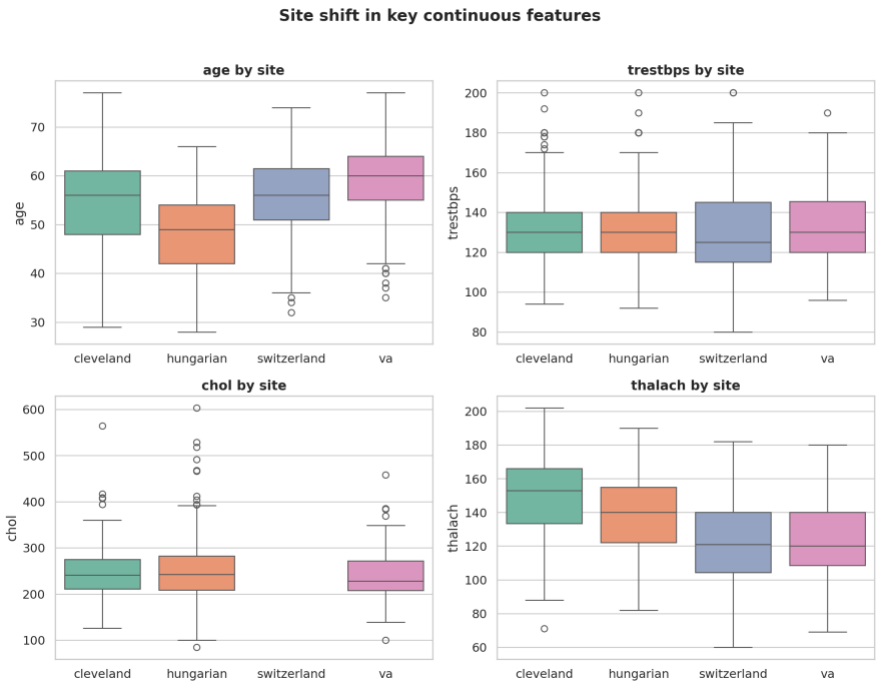


Figure 6: Site shift in key continuous features.

4. Feature Engineering and Modelling

Three models were tuned with RandomizedSearchCV under 5-fold stratified cross-validation: Logistic Regression (an interpretable linear baseline), Random Forest, and HistGradientBoostingClassifier — included specifically because it handles missing values and categorical columns natively, so it receives no imputation at all (unlike the other two, which use a shared ColumnTransformer: median imputation with missing-indicator flags and scaling for numeric columns, constant-sentinel imputation plus one-hot encoding for categorical columns).

The model-selection rule was written down and committed to version control before any hold-out result was ever looked at (docs/DECISIONS.md D3.1): the model with the highest mean cross-validated ROC-AUC wins; ties within 0.005 are broken by preferring the simpler model. ROC-AUC, not a threshold-dependent metric, is the tuning objective, because the operating threshold is chosen separately afterwards.

model	CV ROC-AUC	CV std	Accuracy	Precision	Recall	F1	ROC-AUC	PR-AUC	Brier	Threshold
logreg	0.905	0.011	0.804	0.754	0.961	0.845	0.904	0.915	0.120	0.242
random_forest	0.899	0.019	0.793	0.750	0.941	0.835	0.890	0.885	0.132	0.356
hist_gb	0.894	0.017	0.810	0.760	0.961	0.848	0.898	0.906	0.122	0.260

Table 2: Cross-validated and hold-out metrics for all three tuned models.

Logistic Regression was selected (highest CV ROC-AUC, 0.9055). The operating threshold (0.2419) was chosen from out-of-fold predicted probabilities to meet a 0.95 recall target on the positive (disease) class — missed disease cases are treated as costlier than false alarms, consistent with a screening-tool framing. Logistic Regression's coefficients give a concrete, interpretable check against the site-leakage risk flagged during EDA: the 'ca is missing' category carries an odds ratio of approximately 1 (no effect), while 'ca = 0' carries the single largest coefficient in the model — evidence that ca is being used for genuine clinical signal where known, not as a shortcut for which hospital a row came from.

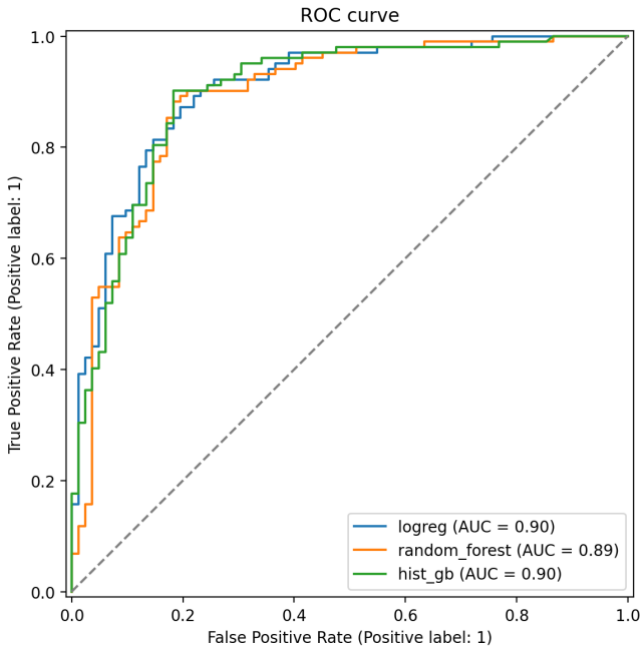


Figure 7: ROC curves for all three models on the hold-out set.

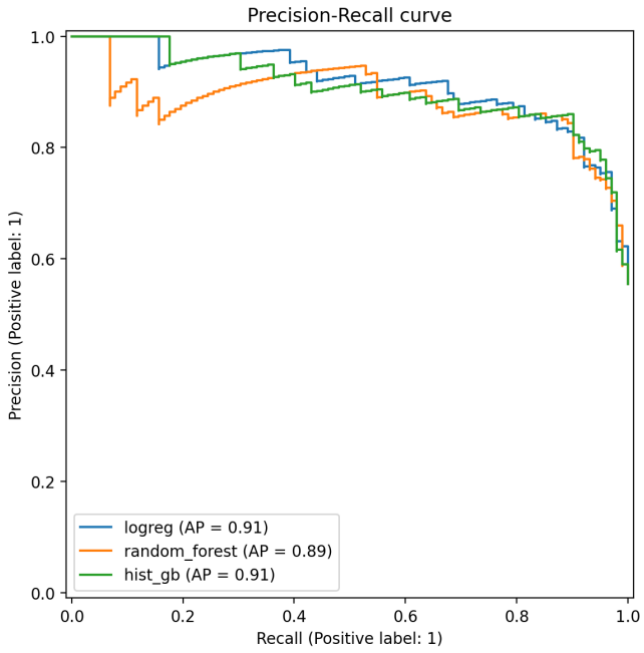


Figure 8: Precision-Recall curves for all three models on the hold-out set.

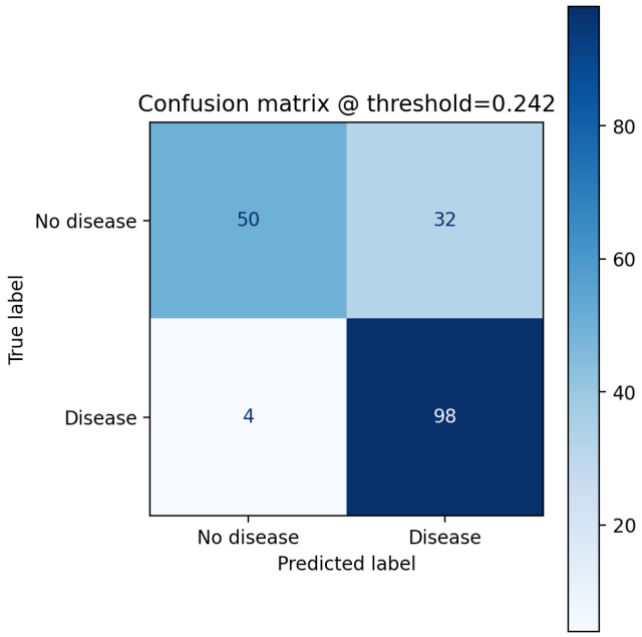


Figure 9: Confusion matrix for the selected model at its operating threshold.

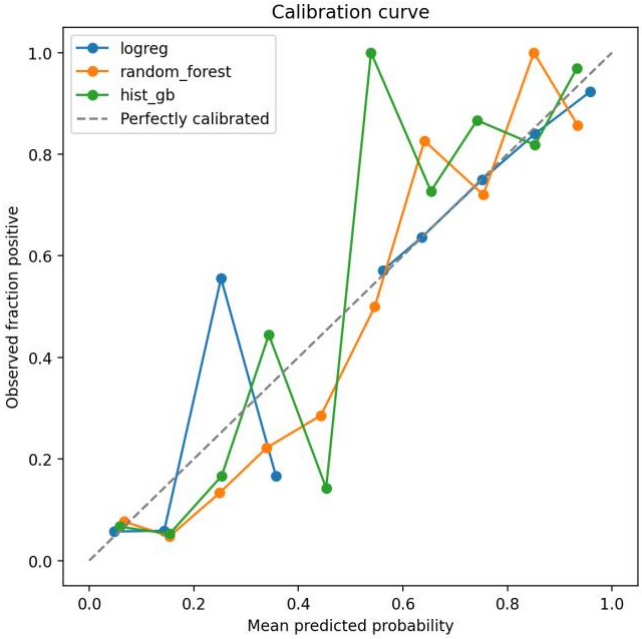


Figure 10: Calibration curves for all three models.

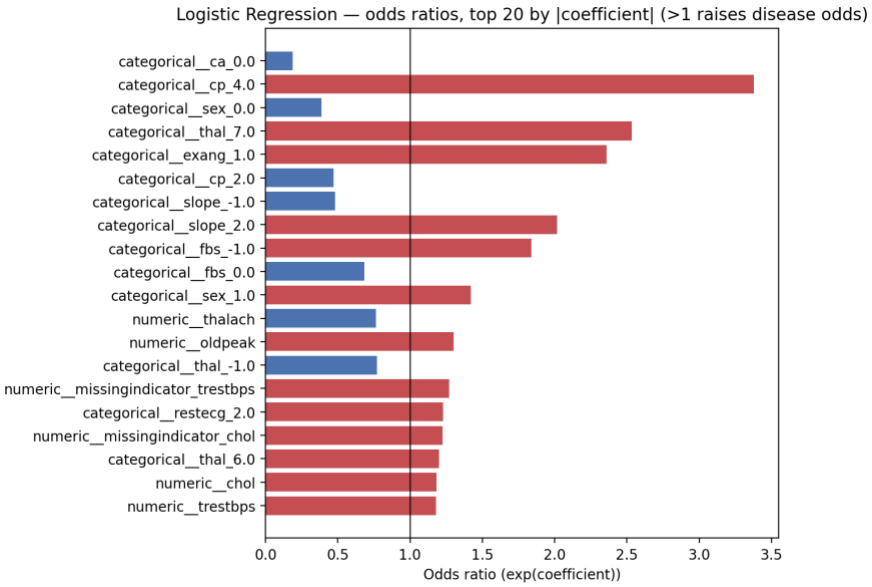


Figure 11: Logistic Regression odds ratios (top 20 by |coefficient|).

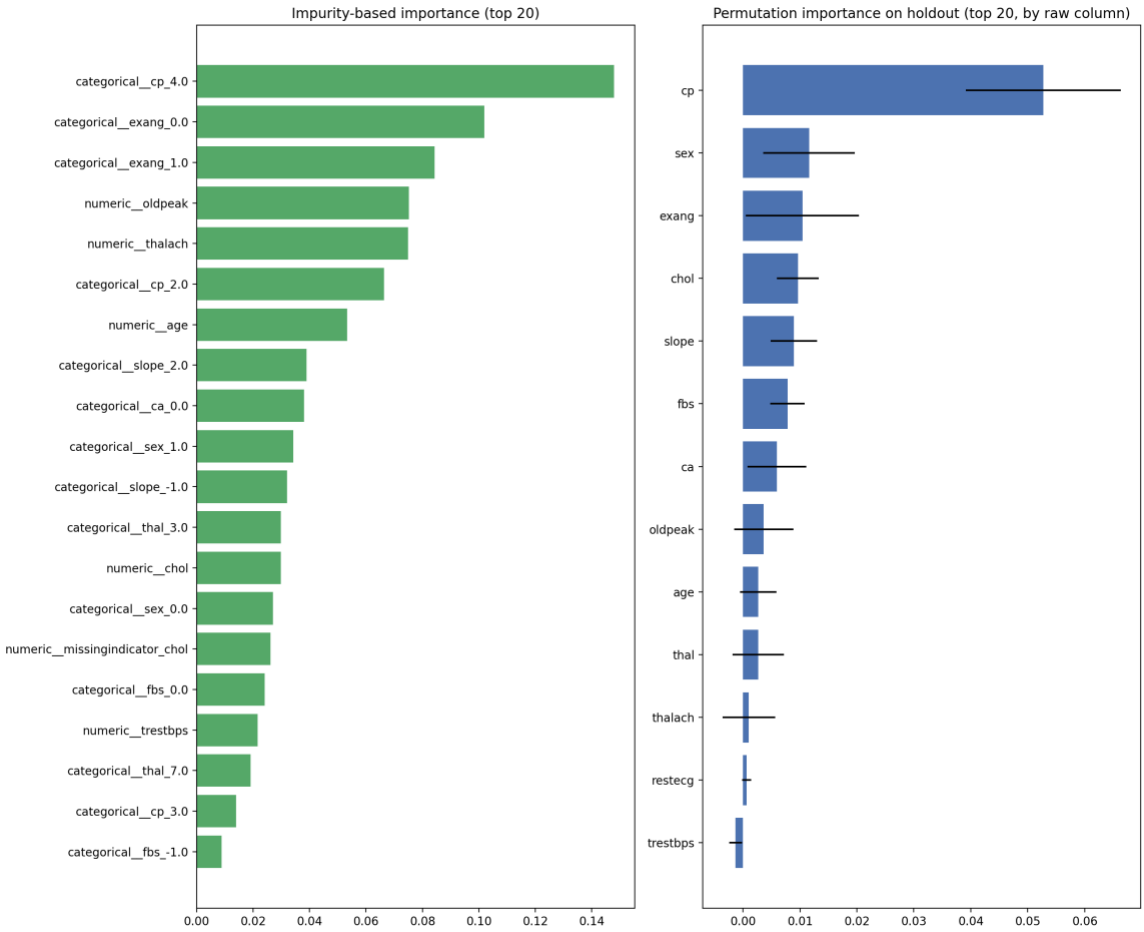


Figure 12: Random Forest impurity-based and permutation importance.

Leave-one-site-out generalisation

The distinctive evaluation for this project trains on three of the four sites and tests entirely on the fourth, in turn, using the single globally-selected threshold throughout (a deployed model does not re-tune itself per incoming hospital).

Held-out site	Train rows	Test rows	Accuracy	Precision	Recall	F1	ROC-AUC	PR-AUC	Brier	Threshold
cleveland	615	303	0.723	0.642	0.892	0.747	0.868	0.863	0.145	0.242
hungarian	625	293	0.662	0.518	0.953	0.671	0.903	0.865	0.134	0.242
switzerland	795	123	0.919	0.965	0.948	0.956	0.726	0.972	0.151	0.242
va	719	199	0.744	0.774	0.926	0.843	0.736	0.869	0.179	0.242

Table 3: Leave-one-site-out metrics, one row per held-out site.

Recall holds at or above the 0.95 target for three of the four sites; Cleveland (0.89) is the exception, likely because it is the only site with near-complete ca/thal/slope data, so a model that never saw Cleveland during training must classify its fuller feature profile using patterns learned mostly from sparser sites. ROC-AUC drops noticeably for Switzerland and VA, consistent with the site-shift seen in Section 3's EDA. Hungarian's precision is low (0.52) when held out — it has the lowest disease prevalence of the four sites, so a threshold tuned on the pooled, higher-prevalence training mix produces more false positives there. This base-rate-shift effect would not be visible from a single pooled k-fold evaluation, which is the entire reason this evaluation exists.

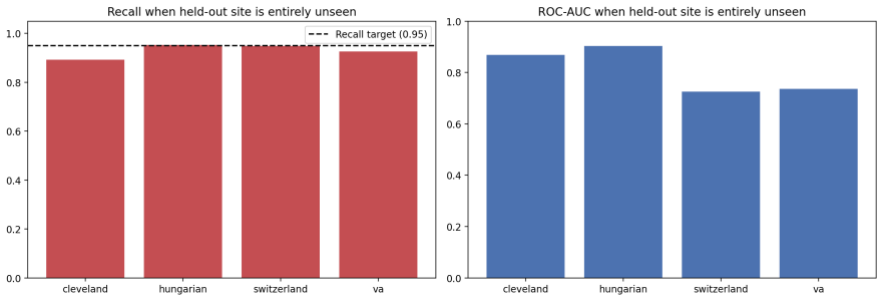


Figure 13: Recall and ROC-AUC by held-out site.

Calibration was checked (Brier score and a reliability curve) but CalibratedClassifierCV changed the Brier score by under 0.001 — not distinguishable from noise on a hold-out set this size — so the selected model's own predict_proba output is shipped directly as the API's confidence score (docs/DECISIONS.md, Phase 3).

5. Experiment Tracking

Every training run logs to a single MLflow experiment (heart-disease-4site) as one parent run with one nested child run per model. Each child run records its hyperparameters, preprocessing choices, cross-validated and hold-out metrics, the full cv_results table, a classification report, and a confusion matrix; the selected model's child run additionally gets a feature-importance plot and the logged sklearn model itself, with an inferred signature and input example. Parent-run artifacts include the model comparison table, ROC/PR/calibration plots, and the model card.

The selected model is registered as heart_disease_classifier and given the alias 'champion' (version 2 at the time of writing) — not MLflow's deprecated Staging/Production stage system. Serving code resolves the model by alias (models:/heart_disease_classifier@champion), never a hardcoded version number, so promoting a new champion never requires a code change. This was verified to be genuinely mutable by running training twice and confirming the alias moved to the new version automatically, with the old version left unaliased for audit and rollback rather than deleted.

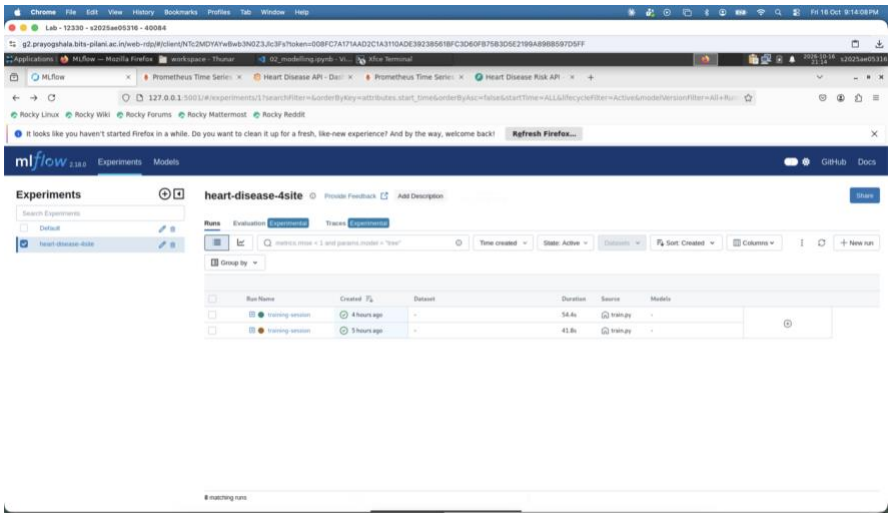


Figure 14: MLflow experiment runs list

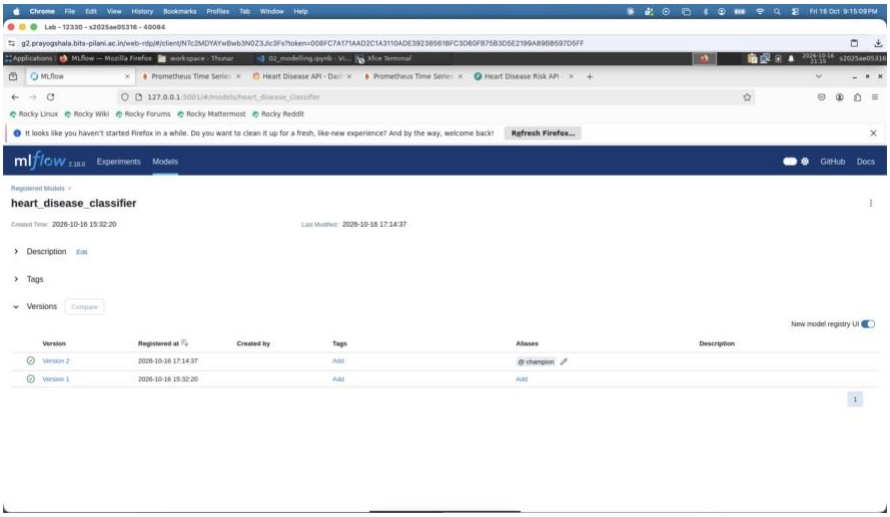


Figure 15: MLflow run detail and registered model

6. Model Packaging and Reproducibility

The trained pipeline is persisted twice: once in the MLflow model format (logged to the registry) and once as models/heart_model.joblib plus models/model_metadata.json. These are deliberately the exact same fitted pipeline, not independently retrained — the joblib artifact is search_results[selected_name].best_estimator_, byte-for-byte the same object passed to mlflow.sklearn.log_model, so the hold-out metrics recorded in the metadata genuinely describe the model being shipped.

The API loads the joblib and metadata pair directly, with no MLflow server reachable at serving time — a deliberate choice, since the model is already fully trained and selected by the time it needs to serve traffic, and depending on a tracking server at that point would be an extra moving part with no benefit.

Field	Value
Model	logreg
Registry version	2
Threshold	0.2419
scikit-learn version	1.5.2
Training data SHA-256	c1bdd6e8cfc99804…
Git commit	fcd0044439a0
Packaged at (UTC)	2026-10-16T11:44:37.544396+00:00

Table 4: Packaged model artifact metadata (models/model_metadata.json).

A reproducibility test (test/test_reproducibility.py) reads the packaged model's own tuned hyperparameters and fits two fresh pipeline instances on the same data, asserting their predictions match to floating-point tolerance — proving the shipped model's determinism without re-running the full ~6-minute hyperparameter search inside a test.

7. CI/CD Pipeline and Automated Testing

68 pytest tests cover data cleaning and download (network calls fully mocked), the preprocessing pipelines (shape, no-NaN-after-transform, unknown-category handling, fit-on-train-only), the model/evaluation/registry code (including one fast end-to-end run of the real training and MLflow-logging path against a tiny synthetic fixture and tiny hyperparameter grids, rather than mocking that path away), and the FastAPI app (health/readiness, the full prediction and batch-prediction paths, six validation-error cases, the Prometheus /metrics endpoint, and a check that raw patient feature values never appear in the logs). Coverage is 93% against a 90% floor enforced by pytest --cov-fail-under.

.github/workflows/ci.yml runs on every push and pull request to main, plus manual dispatch: lint (ruff check and format), test (a matrix of Python 3.12.9 and 3.13.5, mirroring the lab and the development machine exactly), train (downloads and cleans fresh data with a verified fallback to the committed cleaned CSV if UCI is unreachable, trains, and writes a metrics table to the job summary), and docker (builds the image from the repository's committed model artifact and runs the exact same container smoke-test script a developer runs locally).

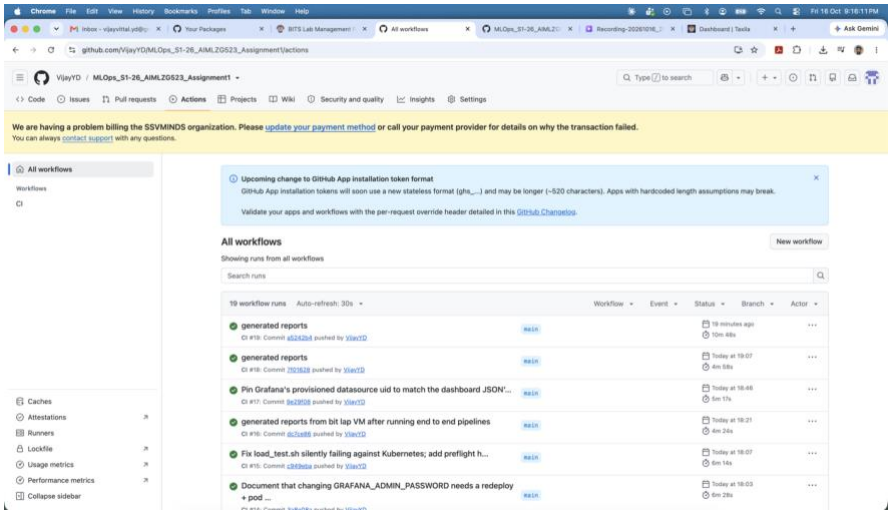


Figure 16: GitHub Actions run with all jobs green

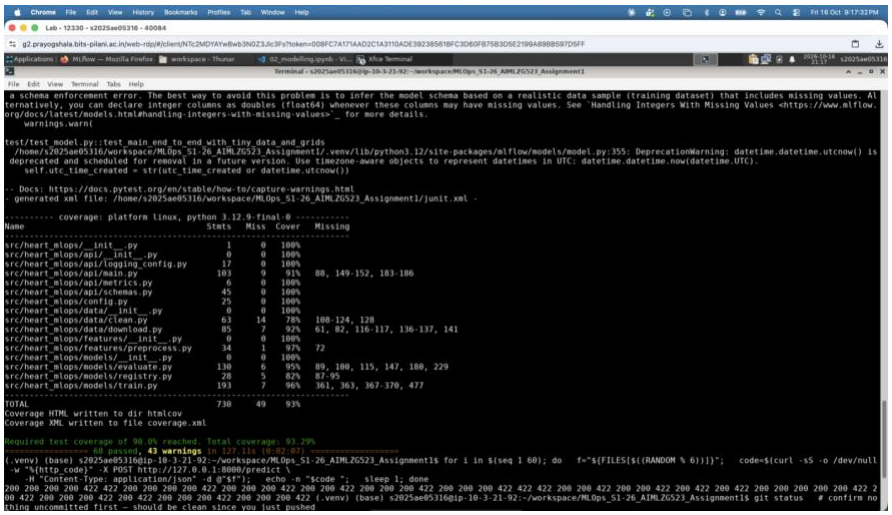


Figure 17: pytest coverage summary

8. Containerisation and API Design

docker/Dockerfile is a multi-stage build on python:3.12.9-slim (matching the lab exactly), built for linux/amd64, running as a non-root user, with only runtime dependencies installed (requirements.txt, not the dev superset), a HEALTHCHECK using the standard library's urllib (so curl does not need to be installed in the image), a configurable worker count, and OCI labels carrying the real build version and git commit SHA.

The FastAPI app (src/heart_mlops/api/) exposes GET /health (liveness), GET /ready (readiness — only once the model artifact has loaded), GET /model-info, POST /predict, POST /predict/batch (up to 100 records per request, sharing the single-record endpoint's row-building logic and using one vectorised predict_proba call), GET /metrics (Prometheus exposition format), and auto-generated OpenAPI documentation at /docs with field-level examples. All thirteen request fields are required, with evidence-based clinical range validation taken from the observed min/max in the training data rather than guessed — out-of-range values or a missing field return HTTP 422, never a 500 or a silent bad prediction. Request logs are structured JSON (request id, path, status, latency, predicted class, model version) and deliberately never include the raw patient feature values, since nothing about serving or debugging predictions requires patient-level clinical data to sit in a log file.

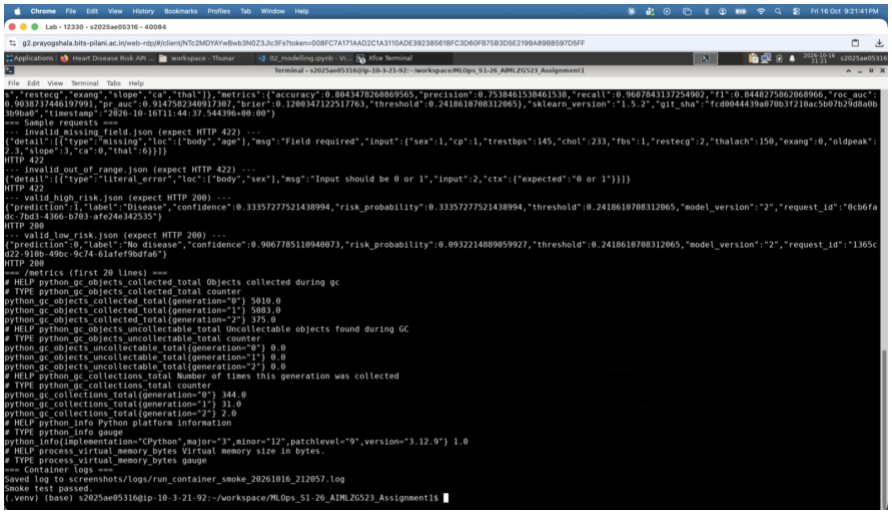


Figure 18: Docker build and container smoke test

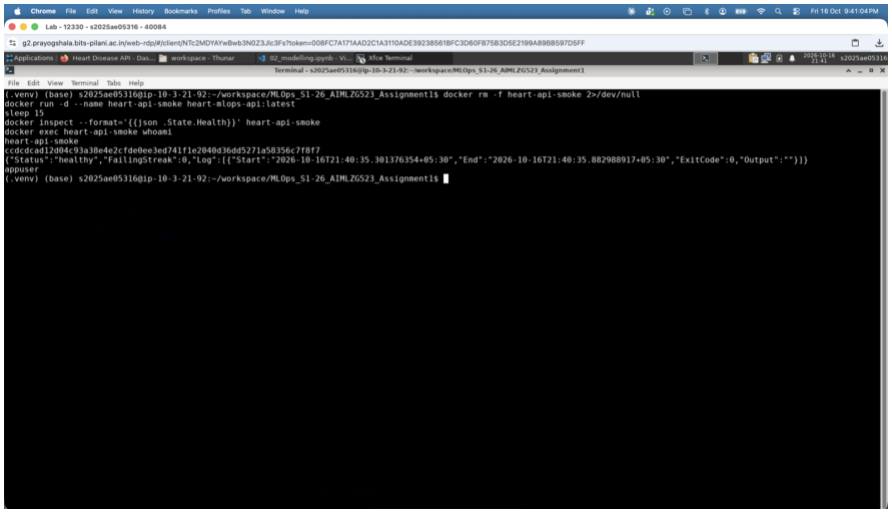


Figure 19: Non-root user and healthy container

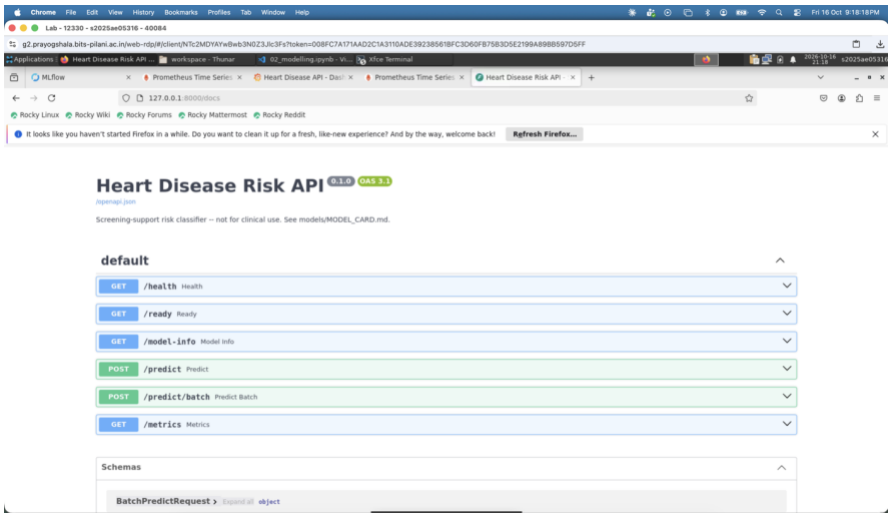


Figure 20: Swagger / OpenAPI documentation

9. Kubernetes Deployment

Deployment manifests are organised with Kustomize (kubectl apply -k, built into kubectl, so Helm is not a dependency): k8s/base/ (namespace, ConfigMap carrying LOG_LEVEL and an optional PREDICTION_THRESHOLD override — both are actually read by the running app, not decorative — a 2-replica Deployment with liveness and readiness probes and a non-root securityContext, a ClusterIP Service, a LoadBalancer Service, an Ingress at host heart-api.local, and a CPU-based HorizontalPodAutoscaler scaling 2 to 5 replicas) and k8s/monitoring/ (Prometheus and Grafana, described in Section 10), composed together by k8s/overlays/minikube/.

scripts/k8s_deploy.sh starts Minikube if needed, enables the ingress and metrics-server addons, builds the image inside Minikube's own Docker daemon (no registry push needed), applies the overlay, and waits for every Deployment to roll out. The Grafana admin password is generated as a Kubernetes Secret from .env, never hard-coded in a committed manifest.

scripts/k8s_verify.sh checks pod/service/ingress/HPA state and then calls /health and /predict through both the Ingress (reached with a Host header, never by editing /etc/hosts) and the LoadBalancer Service. The Ingress is reachable directly at minikube ip on Linux (minikube's ingress addon routes there natively); on macOS it falls back to kubectl port-forward instead of minikube tunnel, since the Ingress's privileged ports (80/443) need an interactive sudo password that an unattended script cannot supply, and the lab disallows sudo entirely in any case. The LoadBalancer Service is different on every OS: a plain type: LoadBalancer Service only gets a real external IP once minikube tunnel is actually running, so the script checks for an assigned external IP first and automatically falls back to kubectl port-forward if there isn't one — no sudo needed either way.

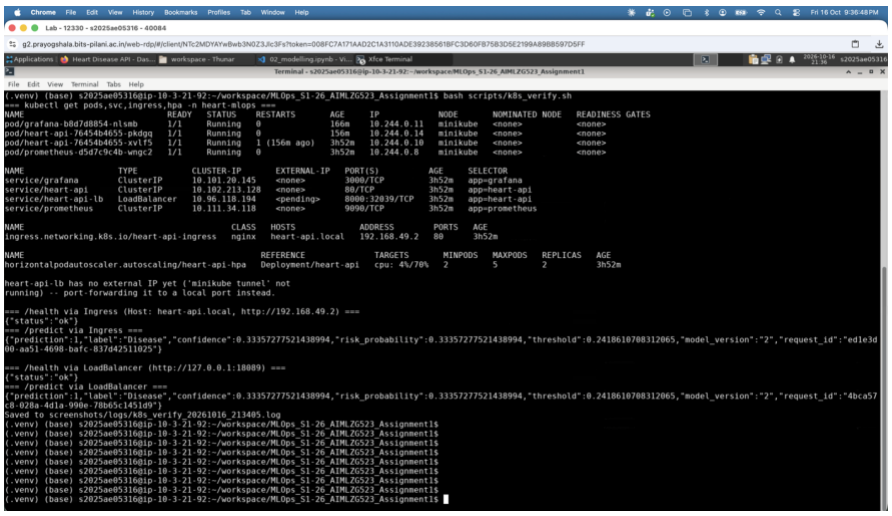


Figure 21: kubectl get pods, services, ingress, HPA, and /predict verified through both the Ingress and the LoadBalancer

10. Monitoring and Logging

The API exposes custom Prometheus metrics: request count by endpoint and status, a latency histogram, prediction count by class, a risk-probability histogram (a simple drift signal), and an inference-error counter. In Kubernetes, Prometheus discovers API pods via the Kubernetes API (pod annotations), not a static target, since pod IPs are not stable; in docker-compose, the same metrics are scraped via a static target. Grafana's datasource and a five-panel dashboard (request rate, p95 latency, predictions by class, risk-probability drift signal, inference error rate) are both provisioned as code — no manual dashboard-building in the UI. Both were verified against their real HTTP APIs, not just 'the containers are up': Prometheus's target for the API reports healthy, a live query returns real prediction counts after running the load test, and Grafana's search API shows the dashboard already present.

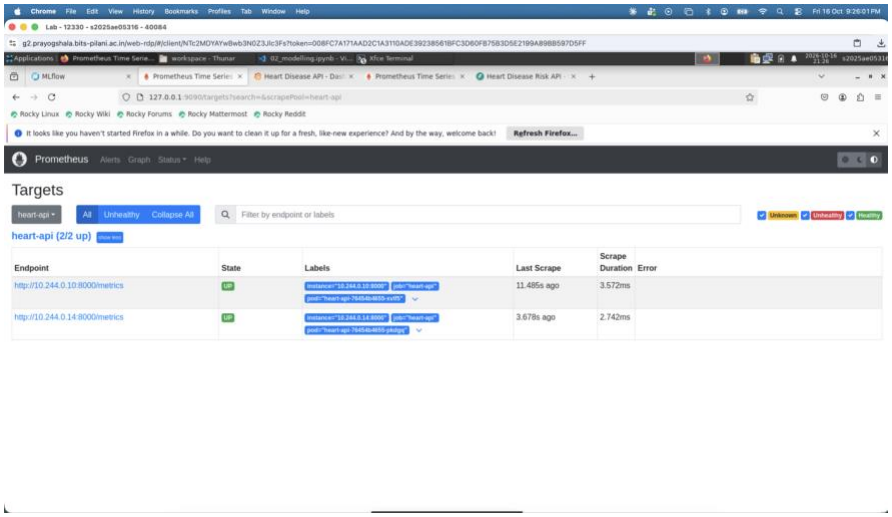


Figure 22: Prometheus targets page

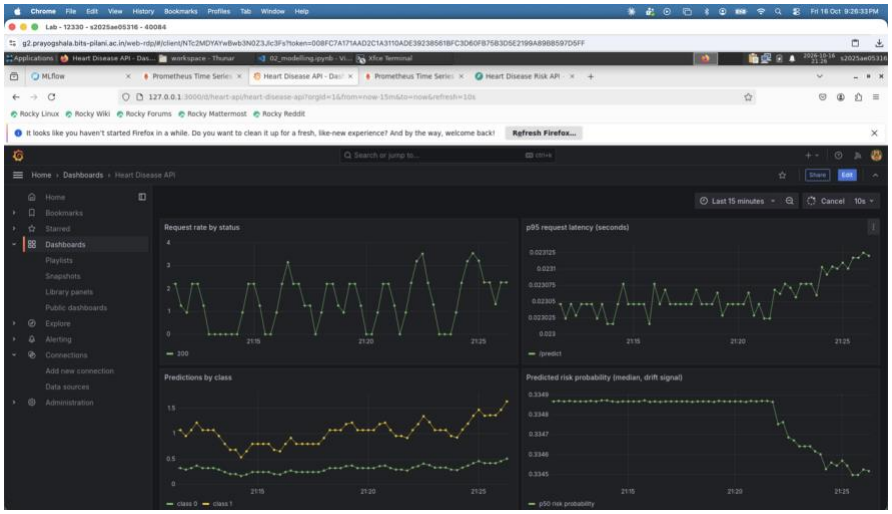


Figure 23: Grafana dashboard with live data

11. Architecture Diagram

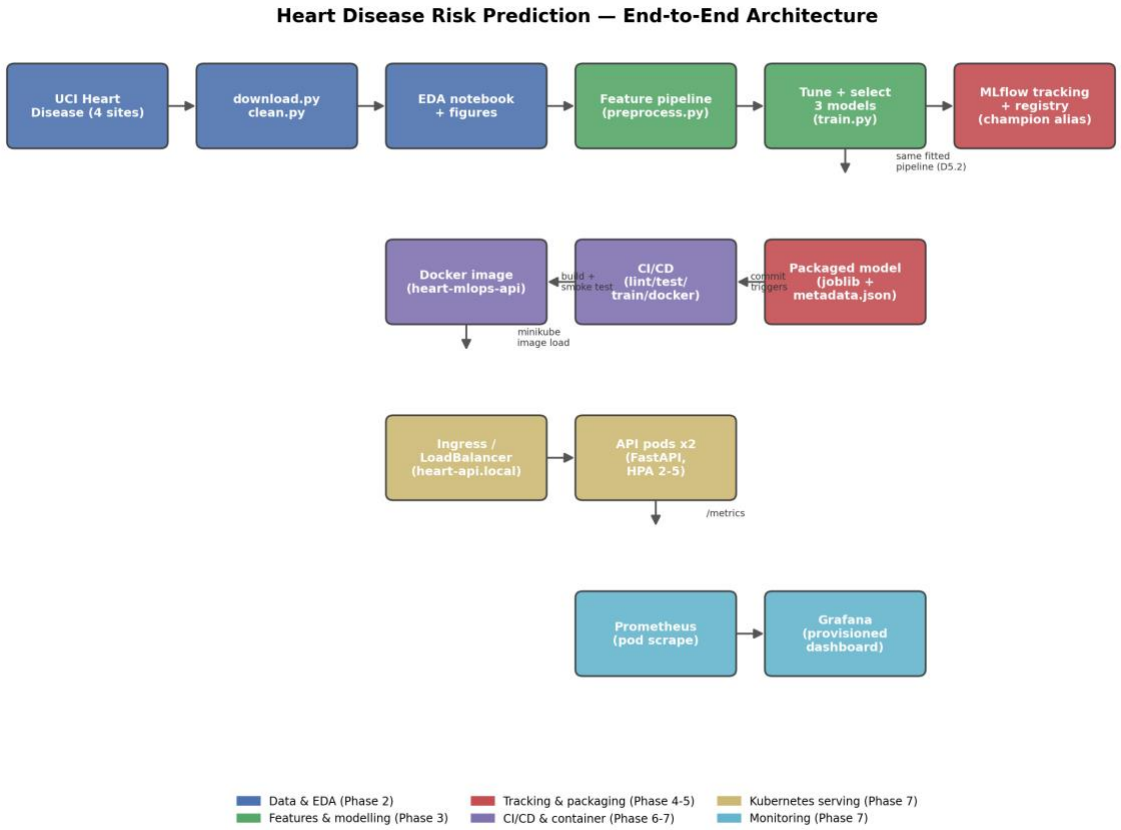


Figure 24: End-to-end architecture, from data acquisition to monitoring.

The full written walk-through of this diagram is in docs/ARCHITECTURE.md.

12. Limitations, Ethical Considerations, and Future Work

This model is a course project artefact, not a validated medical device, and must not be used to make or inform real patient-care decisions (see models/MODEL_CARD.md for the full statement). The training data is small (918 rows) and decades old; ca and thal are over 60% missing outside Cleveland, and the leave-one-site-out results in Section 4 show the model's behaviour on a genuinely unseen hospital is not uniform across sites. The 0.95 recall target was chosen for this project and was not validated against a real clinical cost-benefit analysis. No subgroup fairness analysis beyond the exploratory data analysis in Section 3 has been performed.

The training data's site of origin correlates with factors — era, region, equipment — that could encode bias unrelated to clinical presentation; site is excluded as a model feature

specifically to reduce, though not eliminate, this risk. This model must not be used for insurance, employment, or any decision with legal or financial consequences for a person.

Future work could include: a prospective validation study against current clinical practice; a fairness audit across demographic subgroups; extending the leave-one-site-out evaluation with a genuinely held-out fifth site; and investigating whether a calibration method would help on a larger hold-out set than the one available here.

13. References

- Janosi, A., Steinbrunn, W., Pfisterer, M., & Detrano, R. (1988). Heart Disease [Dataset]. UCI Machine Learning Repository. https://doi.org/10.24432/C52P4X
- Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825-2830.
- MLflow Documentation. https://mlflow.org/docs/latest/index.html
- FastAPI Documentation. https://fastapi.tiangolo.com/
- Kubernetes Documentation. https://kubernetes.io/docs/home/
- Prometheus Documentation. https://prometheus.io/docs/introduction/overview/
- Grafana Documentation. https://grafana.com/docs/grafana/latest/